SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I SAIRAM D 192424111 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SAIRAM D

Annexure A
PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

QUESTION 1

Task: Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.

1. Problem Understanding

The goal is to model the essential exchange that establishes a secure channel between a Client and a Server before any application data is sent. Inputs: a client connection request, the server's digital certificate (containing its public key), and cryptographic parameters proposed by the client. Outputs: a shared, symmetric session key known only to the client and server, and the ability to exchange encrypted application data over that channel. Constraints/assumptions to be modelled: the server's certificate must be validated against a trusted Certificate Authority (CA) before any secret material is trusted; the handshake must abort safely if authentication fails; the session key must never be transmitted in plaintext — it is derived using asymmetric encryption (the server's public key) or a key-exchange primitive so only the two parties can compute it. These inputs, outputs and the trust boundary form the basis of the algorithm below.

2. Algorithm Design (Pseudocode)

```
BEGIN TLS_Handshake_Simulation

// ---------- Data structures ----------
DEFINE ClientHello   = { supportedCipherSuites, clientRandom }
DEFINE ServerHello   = { chosenCipherSuite, serverRandom, serverCertificate }
DEFINE Certificate   = { serverPublicKey, issuerCA, validityPeriod, signature }

FUNCTION Main()
    session <- NULL
    TRY
        clientHello <- ClientRequest()
        serverHello <- ServerRespond(clientHello)

        IF NOT VerifyCertificate(serverHello.serverCertificate) THEN
            LOG("Handshake aborted: certificate verification failed")
            ABORT CONNECTION
        END IF

        premasterSecret <- GeneratePremasterSecret()
        encryptedSecret <- RSA_Encrypt(premasterSecret,
serverHello.serverCertificate.serverPublicKey)
        SEND encryptedSecret TO Server

        sessionKey <- DeriveSessionKey(premasterSecret, clientHello.clientRandom,
serverHello.serverRandom)
        session <- ConfirmHandshake(sessionKey)

        IF session.confirmed THEN
            SecureDataExchange(session, sessionKey)
        ELSE
            LOG("Handshake confirmation failed")
            ABORT CONNECTION
        END IF
    CATCH connectionError
        LOG("Handshake error: " + connectionError.message)
        ABORT CONNECTION
    END TRY
END FUNCTION
```

```
FUNCTION ClientRequest()
    clientRandom <- GenerateRandomNumber()
    RETURN ClientHello(supportedCipherSuites = SUPPORTED_SUITES,
                       clientRandom = clientRandom)
END FUNCTION

FUNCTION ServerRespond(clientHello)
    chosenSuite  <- SelectCipherSuite(clientHello.supportedCipherSuites)
    serverRandom <- GenerateRandomNumber()
    RETURN ServerHello(chosenCipherSuite = chosenSuite,
                       serverRandom = serverRandom,
                       serverCertificate = LOAD_SERVER_CERTIFICATE())
END FUNCTION

FUNCTION VerifyCertificate(cert)
    IF CURRENT_DATE NOT WITHIN cert.validityPeriod THEN
        RETURN FALSE
    END IF
    IF NOT CA_TRUST_STORE.contains(cert.issuerCA) THEN
        RETURN FALSE
    END IF
    IF NOT VerifySignature(cert.signature, cert.issuerCA.publicKey) THEN
        RETURN FALSE
    END IF
    RETURN TRUE
END FUNCTION

FUNCTION DeriveSessionKey(premasterSecret, clientRandom, serverRandom)
    RETURN KDF(premasterSecret, clientRandom, serverRandom)   // key-derivation function
END FUNCTION

FUNCTION SecureDataExchange(session, sessionKey)
    WHILE session.isActive AND MoreDataToSend() DO
        plaintext  <- NextApplicationMessage()
        ciphertext <- AES_Encrypt(plaintext, sessionKey)
        SEND ciphertext TO Peer
        received   <- RECEIVE FROM Peer
        message    <- AES_Decrypt(received, sessionKey)
        PROCESS(message)
    END WHILE
END FUNCTION

END TLS_Handshake_Simulation
```

3. Use of Control Structures

- **IF / ELSE conditionals** gate every trust decision — certificate validity, CA trust, signature check, and handshake confirmation — ensuring the protocol never proceeds on unverified trust.
- **TRY / CATCH** encapsulates the whole handshake so any network or cryptographic failure is caught and safely aborts the connection instead of leaving it in an undefined state.
- **WHILE loop** in SecureDataExchange() models the continuous, bidirectional application-data phase that follows a successful handshake.
- **Modular functions/procedures** (ClientRequest, ServerRespond, VerifyCertificate, DeriveSessionKey, SecureDataExchange) decompose the handshake into single-responsibility units, mirroring the real TLS state machine and improving readability/testability.

4. Pseudocode Structure

The pseudocode uses consistent BEGIN/END blocks, uppercase keywords (IF, WHILE, FUNCTION, RETURN), indentation to show nesting, and inline comments to mark logical sections (data structures, main flow, helper functions). Each function has a single, clearly named responsibility, and the main flow reads top-to-bottom as the actual sequence of handshake events, so it is easy to trace and mark against the rubric.

5. Completeness

The design covers the full lifecycle requested by the question: (1) client request, (2) server authentication via digital certificate — including expiry check, CA-trust check and signature verification, not just a single "is valid" flag, (3) session key generation using a key-derivation function seeded by both parties' random values (preventing replay), and (4) secure, symmetric-key encrypted data exchange in a loop. Failure paths are explicit: an invalid certificate, a failed handshake confirmation, and a runtime connection error each abort safely and log the reason, so the algorithm handles both the happy path and realistic edge cases rather than assuming success.

QUESTION 2

Task: Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.

1. Problem Understanding

The engine must inspect every incoming packet and decide ALLOW or BLOCK by comparing its header fields — source/destination IP address, port number, and protocol (TCP/UDP/ICMP) — against an administrator-defined, ordered rule set. Inputs: an incoming packet (with IP, port, protocol fields) and a rule table where each rule specifies match fields plus an action. Output: a single ALLOW/BLOCK decision, and a log entry for auditing. Key constraints to model: rules must be evaluated in priority order (first match wins, as in real firewalls such as iptables/Windows Firewall); IP matching must support both exact addresses and CIDR/subnet ranges; and — critically — there must be a well-defined default policy (implicit deny) for traffic that matches no rule, since an incomplete rule set must never silently allow traffic.

2. Algorithm Design (Pseudocode)

```
BEGIN Firewall_Rule_Engine

// ---------- Data structures ----------
DEFINE Packet = { srcIP, dstIP, port, protocol }
DEFINE Rule   = { priority, srcIP, dstIP, port, protocol, action }   // action = ALLOW | BLOCK
DEFINE ruleTable   <- LOAD_RULES()          // list of Rule, sorted by priority ascending
DEFINE DEFAULT_POLICY <- BLOCK               // implicit-deny default

FUNCTION FilterPacket(packet)
    FOR EACH rule IN ruleTable DO            // evaluated in priority order
        IF MatchesRule(packet, rule) THEN
            LogDecision(packet, rule.action, rule)
            RETURN rule.action               // first match wins -> stop evaluating
        END IF
    END FOR

    LogDecision(packet, DEFAULT_POLICY, "no matching rule")
    RETURN DEFAULT_POLICY                    // no rule matched -> implicit deny
END FUNCTION

FUNCTION MatchesRule(packet, rule)
    IF rule.srcIP != ANY AND NOT IPInRange(packet.srcIP, rule.srcIP) THEN
        RETURN FALSE
    END IF
    IF rule.dstIP != ANY AND NOT IPInRange(packet.dstIP, rule.dstIP) THEN
        RETURN FALSE
    END IF
    IF rule.port != ANY AND packet.port != rule.port THEN
        RETURN FALSE
    END IF
    IF rule.protocol != ANY AND packet.protocol != rule.protocol THEN
        RETURN FALSE
    END IF
    RETURN TRUE                              // all specified fields matched
END FUNCTION

FUNCTION IPInRange(ip, cidrOrExactIP)
    IF cidrOrExactIP CONTAINS "/" THEN
        RETURN IsWithinSubnet(ip, cidrOrExactIP)     // CIDR match
    ELSE
        RETURN (ip == cidrOrExactIP)                 // exact match
    END IF
END FUNCTION
```

```
FUNCTION LogDecision(packet, action, matchedRule)
    WRITE_LOG(TIMESTAMP(), packet.srcIP, packet.dstIP, packet.port,
              packet.protocol, action, matchedRule)
END FUNCTION

// ---------- Main packet-processing loop ----------
FUNCTION Main()
    WHILE NetworkInterface.hasIncomingPacket() DO
        packet   <- NetworkInterface.readNextPacket()
        decision <- FilterPacket(packet)
        IF decision == ALLOW THEN
            FORWARD(packet)
        ELSE
            DROP(packet)
        END IF
    END WHILE
END FUNCTION

END Firewall_Rule_Engine
```

3. Use of Control Structures

- **FOR EACH loop** walks the priority-ordered rule table with an early RETURN on first match — an efficient short-circuit rather than scanning every rule unnecessarily.
- **Nested IF conditionals** in MatchesRule() check each header field independently (IP, port, protocol) and fail fast, correctly modelling that a rule only applies when every specified field matches.
- **IF/ELSE in IPInRange()** distinguishes CIDR-subnet matching from exact-IP matching, reflecting real firewall rule syntax.
- **An outer WHILE loop** in Main() models continuous, real-time processing of the incoming packet stream, and modular functions (FilterPacket, MatchesRule, IPInRange, LogDecision) separate decision logic from matching logic from logging/audit logic.

4. Pseudocode Structure

Data structures are declared up front (Packet, Rule) so the reader immediately knows what fields are available. Function names are verb-based and self-explanatory (FilterPacket, MatchesRule, IPInRange, LogDecision), indentation marks nesting consistently, and comments call out the two design decisions that matter most for correctness — "first match wins" and "implicit deny" — exactly where they occur in the code.

5. Completeness

The engine handles: (a) exact-IP and CIDR/subnet rules via IPInRange(); (b) wildcard ("ANY") fields so a rule can match on any subset of IP/port/protocol; (c) priority-ordered, first-match evaluation, matching real firewall semantics; (d) a mandatory default-deny fallback so unmatched traffic is never silently allowed — a common real-world security gap that this design explicitly closes; and (e) audit logging on every decision, both allowed and blocked, so the completeness of the security control (not just the filtering logic) is addressed.